

When the Compiler Is Correct but the Package Is Shallow: Evidence on PDF Extraction, Chunking Granularity, LLM Choice, and Observability in the CKF Compiler v1.1

Paulo Tomazinho, PhD
CKF Research
paulo@tomazinho.com.br

May 2026

Abstract

The Compiled Knowledge Format (CKF) proposes to transform human-readable documents into structured, traceable, agent-consumable knowledge packages. Earlier CKF Compiler v1.03.1 studies reported 100% semantic preservation on a 20-unit rubric for the canonical document *What CKF is not (and what it actually is)*, producing 511 counted CKF elements, including 18 retrieval chunks, 66 atomic units, 46 principles, 36 question-answer pairs, and 261 source-traceability entries. However, a subsequent operational test in the public Lab/Compiler environment produced substantially shallower packages from the same PDF, in one case only 13 atomic units, 3 retrieval chunks, 4 entities, and 4 concepts. This paper analyzes the discrepancy not as a failure of the CKF schema or the core v1.1 compiler pipeline, but as evidence of a distinct class of problems: structural degradation before compilation and insufficient pipeline observability. The technical diagnosis identified four primary causes: PDF extraction that removed paragraph breaks and headings; overly large chunk windows; a default LLM optimized for speed rather than deep extraction; and a user-interface label that conflated `protocol_version: ckf-1.0` with the compiler deployment version. The study shows that CKF package quality depends not only on the logical compiler passes - reduce, promote, sanitize, traceability rebuild, and quality gates - but also on preserving document structure before LLM extraction, choosing appropriate chunk granularity, selecting a capable model, and exposing pipeline-level diagnostics. We propose a five-phase correction plan: correct compiler/protocol labeling and expose pipeline trace; preserve structure during PDF extraction; make chunk size configurable and reduce the default; distinguish fast preview models from final-package models; and add regression tests for the canonical CKF reference document. We conclude that CKF quality is end-to-end quality: PDF ingestion, semantic chunking, model selection, compiler passes, traceability, and interface observability must be treated as parts of a single knowledge-compilation system.

Keywords: Compiled Knowledge Format; CKF; knowledge compiler; PDF extraction; semantic chunking; retrieval-augmented generation; provenance; LLM extraction; observability; structured knowledge; AI agents.

1 Introduction

The CKF Compiler has evolved from a shallow structured-output generator into a preservation-oriented pipeline capable of producing rich, traceable, semantically evaluable knowledge packages. Compiler v1.03.1, later stabilized publicly as v1.1, corrected the over-sanitization regression observed in v1.03 by introducing a field-aware sanitizer, preserving retrieval chunks, rebuilding source traceability, and maintaining language and truncation filters.

A later operational experiment revealed an important phenomenon. The same canonical document used in the v1.03.1 study, *What CKF is not (and what it actually is)*, produced a much shallower package when compiled through the public Lab/Compiler interface. The difference was not explained by an

old compiler route or by a broken sanitation module. The technical diagnosis showed that the Lab already called `compileToCkf`, which executed the canonical `runCkfPipeline v1.1`, also used by the compiler route, MCP, and administrative recompilation.

This finding shifts the problem. The logical compiler was correct, but the textual representation delivered to the compiler did not preserve enough document structure to support rich extraction. In other words, the failure was not only in what the compiler did after receiving text; it was in how the PDF was converted into text, segmented into chunks, and sent to the model.

This paper documents that finding as an important step in CKF’s technical evolution. It argues that CKF package quality depends on the whole chain:

source document → structure-preserving text extraction → semantic chunking → LLM extraction → reduce → promotion → sanitation → traceability rebuild → quality gates → UI observability.

If the early stages degrade structure, the downstream compiler can be technically correct and still produce a shallow package.

2 Expected Behavior from CKF Compiler v1.03.1

The internal v1.03.1 benchmark used the canonical document *What CKF is not (and what it actually is)*. The resulting CKF package achieved full preservation on a 20-unit semantic rubric and produced the structural population summarized in Table 1.

Metric	v1.03.1 benchmark result
Semantic preservation	100%
Counted CKF elements	511
retrieval_chunks	18
atomic_units	66
principles	46
decision_rules	6
procedures	4
anti_patterns	25
exceptions	15
playbooks	3
qa_pairs	36
source_traceability	261
__saq	1.0
human_readability	0.8
ai_utility_score	1.0

Table 1: Expected rich output from the internal CKF Compiler v1.03.1 benchmark.

The document is especially suitable for diagnostic testing because it is explicitly contrastive. Its sections include: CKF is not a vector database; CKF is not a knowledge graph or graph database; CKF is not an ontology; CKF is not a fine-tuning system; CKF is not a replacement for RAG; CKF is not a replacement for GraphRAG; CKF is not a replacement for MCP; CKF is not a document format for humans; CKF is not a prompt library or system prompt format; and So what is CKF? Each section naturally supports atomic units, retrieval chunks, principles, anti-patterns, exceptions, question-answer pairs, and, in some cases, procedures or playbooks.

3 Empirical Observation: Shallow Output in the Lab and Compiler

When the same PDF was compiled in the public Lab interface, the output was substantially shallower. One observed result contained the section counts in Table 2.

Metric	Observed shallow Lab output
atomic_units	13
retrieval_chunks	3
if_then_rules	0
decision_rules	3
procedures	0
heuristics	0
entities	4
concepts	4
source_traceability	59

Table 2: Observed shallow output from the Lab interface.

A subsequent output generated through the compiler route improved over the Lab result, but still remained far from the v1.03.1 benchmark. Table 3 compares the observed compiler output with the benchmark.

Section	Observed compiler output	v1.03.1 benchmark
entities	4	12
concepts	5	17
principles	10	46
decision_rules	1	6
procedures	1	4
anti_patterns	6	25
exceptions	6	15
mental_models	0	2
playbooks	1	3
qa_pairs	10	36
retrieval_chunks	5	18
atomic_units	12	66
source_traceability	62	261
Total counted	124	511

Table 3: Observed compiler output compared with the v1.03.1 benchmark.

The observed compiler package contained approximately 24% of the structural population of the benchmark package. It preserved some important ideas, but did not reproduce the richness of the v1.03.1 study.

4 Research Question

This discrepancy led to the following research question:

Why does the same canonical document produce a much shallower CKF package in the public Lab/Compiler environment than in the v1.03.1 benchmark, even though the canonical v1.1 pipeline is being called?

Two broad hypotheses were possible:

1. **Compiler-regression hypothesis:** the v1.1 pipeline was broken or an old compiler route was running.
2. **Pre-compilation degradation hypothesis:** the pipeline was correct, but the text delivered to it had lost structure before extraction.

The technical diagnosis supported the second hypothesis.

5 Technical Diagnosis

The diagnosis identified four real causes and one interface-level communication problem.

5.1 Cause 1: PDF Extraction Removed Document Structure

The highest-impact cause was PDF extraction. The PDF reader joined `PDF.js content.items` using a single space. As a result, each page became a long line with no paragraph breaks, no headings, and little visual structure. This is critical because `chunkSemantically` relies on paragraph breaks and heading patterns to partition a document into meaningful spans.

When such breaks disappear, the chunker cannot see headings such as:

CKF is not a vector database; CKF is not a knowledge graph or graph database; CKF is not an ontology; CKF is not a fine-tuning system.

Instead, the document becomes one or a few long blocks. With `APPROX_CHARS_PER_CHUNK = 12_000`, a 14,000-character PDF will typically produce only one to three large chunks. In the internal v1.03.1 study, by contrast, the same content was compiled from clean Markdown with preserved headings, producing roughly 12 to 18 semantic sections and enabling 18 retrieval chunks.

The key finding is:

Loss of headings and paragraph structure during PDF extraction reduces semantic granularity before the compiler begins operating.

5.2 Cause 2: Chunk Size Was Too Large

The chunker used a default ceiling of 12,000 characters. This is safe for token limits but too large for dense extraction. Large chunks produce two negative effects: the model tends to summarize instead of extracting comprehensively, and downstream promotion receives too few atomic units and retrieval chunks to generate rich principles, anti-patterns, question-answer pairs, procedures, and playbooks.

The v1.03.1 benchmark used smaller chunks in the range of roughly 5,000 to 6,000 characters. Reducing the default to approximately 6,000 characters, or making chunk size configurable, is therefore a reasonable correction.

5.3 Cause 3: The Default Model Was Optimized for Speed

The public environment used `google/gemini-3-flash-preview` through the Lovable provider. Flash-style models are useful for fast preview, but tend to produce compact extractions. The v1.03.1 benchmark used stronger models, such as GPT-5 or Gemini Pro-class systems, better suited to comprehensive structured extraction.

Therefore, 13 atomic units for a 14,000-character PDF is plausible for a fast preview model, but not for a final deep compilation mode. The interface should distinguish fast preview from final package generation.

5.4 Cause 4: The UI Conflated Protocol Version and Compiler Version

The interface showed a label similar to:

Compiler engine - CKF v1.0

This mixed two distinct layers: `protocol_version: ckf-1.0` and `compiler_deployment_version: v1.1`. The diagnosis showed that `metrics.compilerVersion = "v1.1"` was already returned by `runCkfPipeline`, but the UI ignored that value.

The interface should display them separately:

Compiler deployment: CKF Compiler v1.1; Protocol: ckf-1.0; Provider: Lovable; Model: google/gemini-3-flash-preview.

This distinction matters because schema compatibility does not imply that the compiler engine is old.

5.5 Cause 5: Missing Pipeline Observability

The fifth issue was not the direct cause of shallow extraction, but it made the problem hard to diagnose. The interface did not expose the number of chunks, partials, failed chunks, counts before and after reduce, counts before and after promotion, counts before and after sanitation, promoted items, removed items, model choice, chunk size, or unique source references.

Without this observability, the user cannot determine whether the loss occurred in PDF extraction, chunking, LLM extraction, reduction, promotion, sanitation, traceability rebuilding, or UI rendering.

The diagnosis found that `runCkfPipeline` already returned several relevant signals, including the package, quality metrics, promotion audit, sanitizer audit, warnings, and compiler version. The missing step was exposing those signals in the UI.

6 Scientific Interpretation

The main finding is:

A compiler can be correct and still produce a shallow package if document structure is destroyed before compilation.

This introduces a new dimension in CKF evaluation: *input structural fidelity*. Earlier studies focused on semantic preservation, section allocation, language fidelity, sanitizer precision, retrieval readiness, and traceability consistency. This study adds four upstream dimensions: input structural fidelity, chunking granularity, model extraction depth, and pipeline observability.

A PDF converted into text without paragraphs or headings is not equivalent to clean Markdown. Although both may contain similar words, they do not expose the same structure to the chunker or to the LLM. For a knowledge compiler, headings are semantic signals, not decoration.

7 Headings as Semantic Signals

The reference document is organized as a sequence of contrastive headings. Each heading represents a core distinction. If headings are preserved, the chunker can create semantic spans. Each span can then generate a coherent cluster of CKF units.

If headings are erased, the document becomes a long textual mass. The LLM must infer conceptual boundaries by itself. With fast models, this tends to produce compact summarization rather than comprehensive extraction.

Thus:

Headings are implicit semantic metadata and must survive PDF extraction.

8 Chunking Granularity

Chunking granularity directly affects the number and quality of extracted units. Overly large chunks reduce the number of extraction calls, mix unrelated sections, weaken source-reference granularity, encourage summarization, and reduce the effectiveness of downstream promotion.

Smaller, semantically aligned chunks isolate topics, improve coverage, generate more useful source references, facilitate deduplication, and increase retrieval chunks and question-answer pairs. The chunker is therefore not merely a cost optimization; it is a semantic stage of the compiler.

9 LLM Choice

The diagnosis also shows that LLMs are not interchangeable when the goal is deep compilation. Fast models are suitable for preview, pipeline testing, cheaper compilation, short documents, and exploration. Final packages require models better able to extract multiple section classes, preserve contrasts, generate extensive question-answer pairs, maintain source excerpts, avoid shallow summarization, and follow deep-extraction instructions.

The interface should therefore make the trade-off explicit:

Use Flash for fast preview. Use Pro or a stronger model for final packages.

10 Compilation Modes

The findings suggest three practical compilation modes, summarized in Table 4.

Mode	Characteristics	Appropriate use
Fast preview	Flash model, large chunks, compact extraction	Quick inspection
Standard	Medium chunks, intermediate model	General compilation
Deep extraction	Preserved headings, smaller chunks, stronger model	Final packages, benchmarks, public documentation

Table 4: Suggested compilation modes for CKF.

The error in interpretation was treating a preview or standard output as equivalent to a deep benchmark output.

11 Recommended Implementation Plan

11.1 Phase 1: Correct UI Labeling and Add Observability

The UI should clearly separate compiler deployment version, protocol version, provider, and model. It should add a collapsible pipeline trace showing chunks, failed chunks, token counts, duration, section counts, promotion audit, sanitizer audit, unique source references, and warnings. This transforms the UI from a result renderer into a diagnostic instrument.

11.2 Phase 2: Preserve Structure During PDF Extraction

The PDF reader should reconstruct lines and paragraphs using PDF.js item coordinates. It should group items by visual line, insert line breaks and paragraph breaks based on vertical distance, filter repeated headers and footers, detect heading candidates, prefix detected headings with Markdown heading markers, and retain a fallback path for difficult layouts.

11.3 Phase 3: Make Chunk Size Configurable

The default chunk size should be reduced to approximately 6,000 characters, or exposed as an advanced option with small, medium, and large settings. Users should be told that smaller chunks can improve package quality at higher cost.

11.4 Phase 4: Guide Model Selection

The interface should distinguish preview from final compilation. Flash models should remain available, but stronger models should be recommended for final packages and benchmark-grade outputs.

11.5 Phase 5: Add Canonical Regression Tests

The canonical reference document should become a regression fixture. Minimum thresholds should include nonzero and reasonably high retrieval chunks, atomic units, and traceability, with no orphan source-traceability references. A deep benchmark can enforce stricter thresholds for retrieval chunks, atomic units, question-answer pairs, unique source references, and traceability entries.

12 Implications for CKF Evaluation

The earlier evaluation model was:

source → compiler → CKF package.

This study suggests a more accurate model:

source document → ingestion fidelity → semantic chunking → extraction model → compiler passes → package quality → UI observability.

Future CKF studies should therefore record source format, extraction method, extracted character count, preserved heading count, paragraph count, chunk count, average chunk size, model, provider, extraction mode, counts by stage, final counts, and unique source references. Without these metadata, two CKF packages generated from the same PDF may not be comparable.

13 Relationship to Earlier CKF Studies

Earlier studies established that CKF-0.1 failed by shallow summarization; v1.0 preserved semantics but had weak section allocation; v1.01 improved promotion and section allocation; v1.02 introduced metadata calibration and filters; v1.03 introduced global sanitation but over-sanitized; and v1.03.1 corrected over-sanitization through a field-aware sanitizer.

The present paper adds a new conclusion:

Even with a correct compiler v1.1 pipeline, package quality depends on structural fidelity during ingestion and on chunking granularity.

Thus, CKF's evolution must include not only internal compiler passes, but also the document pre-compiler.

14 Recommendations for the CKF Project

The PDF extraction layer should be versioned, tested, and measured as part of the compiler. Pipeline trace should be exposed by default. UI labels should never conflate `ckf-1.0` protocol compatibility with compiler deployment version. Deep extraction mode should be available for final packages. Each package should record source format, extraction method, heading count, paragraph count, chunk count, average chunk size, model, provider, and extraction mode.

15 Threats to Validity

This study is based on a single canonical document. Although the document is dense and diagnostically useful, other documents may behave differently. Two-column PDFs, scanned documents, legal layouts, tables, and slide decks may require specialized extraction strategies. In addition, the comparison between the v1.03.1 benchmark and public Lab output involves differences in model, input format, and chunking configuration. Therefore, the correct conclusion is not that CKF failed as a format or that compiler v1.1 failed as a pipeline. The conclusion is that ingestion and extraction must be controlled and observable.

16 Conclusion

The investigation showed that the Lab/Compiler was not necessarily running an old compiler. The canonical v1.1 pipeline was present, but shallow output was caused by upstream or external factors: PDF extraction that destroyed structure, overly large chunk windows, a default model optimized for speed, and insufficient observability in the UI.

This finding is important for CKF’s scientific maturity. Knowledge compilation does not begin at `reduce()` and does not end at `sanitize()`. It begins with preserving the structure of the source document.

The central conclusion is:

CKF quality is end-to-end quality.

CKF package quality is a property of the entire chain: document, extraction, chunking, model, compiler passes, traceability, and interface. The discrepancy observed in the Lab therefore becomes evidence for the next stage of CKF development: treating PDF ingestion, semantic chunking, and pipeline observability as essential components of the knowledge compiler itself.

Data and Software Availability

The materials related to this study include the canonical PDF *What CKF is not (and what it actually is)*, the benchmark v1.03.1 CKF packages, the observed Lab and Compiler outputs, the technical Lovable diagnosis, the CKF Compiler v1.1 source code, and the v1.03.1/v1.1 regression tests.

Conflict of Interest Statement

The author is affiliated with CKF Research, the organization responsible for developing the Compiled Knowledge Format and CKF Compiler. This study documents both successes and limitations observed during the development of the technology.

Author Contributions

Paulo Tomazinho conceived CKF, conducted the preservation and comparison studies, supplied the artifacts, analyzed the outputs, and consolidated the technical recommendations for compiler evolution.